MedVoice: A Voice-First, Multilingual Medicine-Reminder Assistant for Older Adults using IndicTrans2 and a Small Language Model with Rule-Based Validation

Primary Track: F — Applied AI Systems Secondary Track: E — Neuro-Symbolic Reasoning

Team No.: [TO FILL]

Team Members (maximum three)

Reg. No.	Name	Primary role in the project
24BDS0212	Laksh Ahuja	Sole member: entire system — React front end, FastAPI back end, IndicTrans2 + Qwen pipeline, validation, scheduler, MongoDB, evaluation and report

Report submitted for the

Final Project Review of

Course Code: BCSE306 — Artificial Intelligence

Slot: D1

Professor: Dr. Ilanthenral Kandasamy

GitHub Repository: https://github.com/laksh-ahuja06/local-language-ai-chatbot

Vellore Institute of Technology, Vellore

Fall Semester 2026–27 | October 2026

Declaration and Individual Contribution

I declare that this report and the accompanying code are my own work, carried out for BCSE306 Artificial Intelligence during Fall Semester 2026–27. All sources, datasets, pre-trained models and AI tools used have been acknowledged. Every reference cited has been personally checked by me. I understand that I may be examined individually on any part of this work during the viva.

Reg. No.	Name	Modules / components owned	Report sections written	Contribution (%)	Signature
24BDS0212	Laksh Ahuja	All modules: speech UI and language slider (React), FastAPI API and pipeline, IndicTrans2 translation, Qwen extraction and prompt, validation rules, APScheduler scheduler and notifications, MongoDB storage, evaluation script and test set	All sections	100%	

Abstract

Older adults in India often manage several daily medicines, yet most reminder apps assume English text input. This project presents MedVoice, a voice-first web app that lets a user say a reminder in Hindi, Tamil, Telugu or Bengali and turns it into a scheduled notification. The request is translated to English with IndicTrans2, a 0.5-billion-parameter language model (Qwen2.5-0.5B-Instruct) extracts the medicine, dose, time and frequency, and a rule-based validation layer normalises the output and accepts it only if the scheduler can execute it; everything runs locally on an 8 GB laptop. On a new parallel test set of 100 requests (25 per language) with hand-written gold labels, the system produced a fully correct reminder for 52% of requests (95% CI [42, 62]), compared with 42% for a rule-based parser and 13% for the original zero-shot 1.5B pipeline, and every request returned a valid reminder or a clear error, with extraction taking 1.7 s on average. Ablations show that translation is essential (0% without it) and that the structured prompt and validation account for most of the gain over the original design. Error analysis traced the largest single error source to a time-formatting bug in the translation toolkit's preprocessing and to transliterated drug names; fixing the time bug afterwards raised accuracy to 57% on the same, already-seen test set. The results fall short of the 70% target, so the system is a promising but not yet deployable prototype.

Keywords: medication reminders; older adults; Indian languages; machine translation; small language models; neuro-symbolic validation

Table of Contents

Declaration and Individual Contribution 1
Abstract 1
Table of Contents 1
1. Introduction 1
1.1 Background and Motivation 1
1.2 Problem Statement 1
1.3 Research Questions / Hypotheses 1
1.4 Contributions 1
1.5 Organisation of the Report 1
2. Literature Review 1
2.1 Search Strategy 1
2.2 Literature Summary Table 1
2.3 Synthesis and Research Gap 1
3. Objectives of the Project 1
4. Innovation Component and System Design 1
4.1 Innovation Component 1
4.2 Positioning Against the Closest Existing Work 1
4.3 System Architecture 1
4.4 Module Description 1
4.5 AI Techniques Used and Design Rationale 1
5. Methodology and Implementation 1
5.1 Data 1
5.2 Preprocessing 1
5.3 Models Used 1
5.4 Prompting, Fine-tuning and Retrieval Configuration 1
5.5 Experimental Setup and Hyperparameters 1
5.6 Compute and Cost 1
5.7 Tools and Frameworks 1
5.8 Implementation, Screenshots and Demo 1
6. Evaluation Protocol 1
6.1 Evaluation Data and Split Hygiene 1
6.2 Metrics 1
6.3 Baselines 1
6.4 Human Evaluation and LLM-as-a-Judge (if used) 1
6.5 Statistical Analysis 1
7. Results and Discussion 1

7.1 Main Results....1
7.2 Comparative Analysis....1
7.3 Ablation Study....1
7.4 Error Analysis....1
7.5 Answers to the Research Questions....1
7.6 Threats to Validity and Limitations....1
8. Responsible AI and Ethical Considerations....1
8.1 Risk Assessment....1
8.2 Data Privacy and Consent....1
8.3 Fairness....1
8.4 Safety, Security and Misuse....1
8.5 Licences and Attribution....1
9. Conclusion and Future Work....1
10. Reproducibility Checklist....1
11. References....1
12. Declaration of Generative AI Use....1
Appendices....1
Annex A: Track-Specific Requirements....1
Track A — Agentic and Multi-Agent LLM Systems....1
Track B — Retrieval-Augmented Generation, Grounding and Hallucination....1
Track C — Bias, Fairness and Sociotechnical Auditing....1
Track D — AI Security, Robustness and Trust....1
Track E — Neuro-Symbolic, Fuzzy and Neutrosophic Reasoning....1
Track F — Applied AI Systems: Perception, Decision Support, Simulation and Robotics. .1
Annex B: Indicative Track Assignment....1
Annex C: Final Review Evaluation and In-Person Viva....1
Indicative evaluation scheme....1
What to bring to the viva....1
What the viva will examine....1

1. Introduction

1.1 Background and Motivation

Taking medicines on time is one of the most common self-management tasks of older adults, and one of the easiest to get wrong. Reminder apps help: a 2025 meta-analysis of 14 randomised trials found that app-based interventions raised adherence on the 8-item Morisky scale by 0.57 points (95% CI 0.33–0.80) and percentage adherence by 18.9 points [14]. However, most of these apps assume a person who can read English menus, type, and navigate small touch targets.

Older adults increasingly prefer voice over touch screens because of declining eyesight and dexterity, and co-design studies with them ask for assistants that are personalised and set up reminders through natural conversation [11]. At the same time, a 2025 systematic review of 72 papers found low acceptance of health conversational agents among older adults, driven by unstable behaviour, privacy concerns and loss of independence [13]. In a month-long in-home study, voice-assistant errors were a major source of breakdowns for older users [12].

In India the problem is compounded by language. Speech and translation technology for the 22 scheduled languages has improved rapidly [1], [3], [4], but general-purpose language models remain far stronger in English than in Indian languages [5]. A reminder assistant for an elderly Tamil or Bengali speaker therefore has to understand a spoken request in that language, convert it into an exact time and repetition rule, and do so reliably on modest hardware, because a wrongly scheduled medicine is worse than no reminder at all.

1.2 Problem Statement

Given a short spoken request in Hindi, Tamil, Telugu or Bengali (for example, "remind me to take Crocin at 9 pm every day"), the system must produce a reminder record {medicine, dose, time, frequency} that the scheduler can execute without human correction, store it, and fire a notification at the right time. The extraction must work on an ordinary 8 GB laptop without paid cloud APIs, must never crash on malformed model output, and must be measurably accurate in all four languages. The core technical problem studied in this report is the extraction step: turning a free-form request in an Indian language into a correct, schedulable reminder.

1.3 Research Questions / Hypotheses

RQ1 (accuracy). How accurately does the full MedVoice pipeline (IndicTrans2 translation → Qwen2.5-0.5B with a chat-format few-shot prompt → rule-based validation) produce correct,

schedulable reminders, compared with (i) a rule-only baseline and (ii) the original pipeline (Qwen2.5-1.5B, zero-shot plain prompt, defaults-only validation)?

RQ2 (components). How much does each component contribute: translation to English, the system prompt, the few-shot examples, the rule-based frequency step and the validation layer as a whole?

RQ3 (fairness and robustness). Is accuracy similar across the four languages, and how much does it drop when the input contains speech-recognition-style errors or is translated from the wrong source language?

RQ4 (efficiency). Can the 0.5B model match the 1.5B model in accuracy while being fast enough for interactive use on an 8 GB laptop?

Hypotheses (stated before running the evaluation): H1 — the full pipeline is significantly more accurate ($p < 0.05$) than both baselines; H2 — removing translation or the few-shot examples causes the largest drops; H3 — no language differs significantly from the others; H4 — 0.5B is within 10 percentage points of 1.5B at under half the latency.

1.4 Contributions

- A working voice-first, four-language medicine-reminder app (React, FastAPI, MongoDB, APScheduler) that runs entirely on an 8 GB laptop with no API cost (Sections 4–5).
- A neuro-symbolic extraction pipeline in which rules and the scheduler's own parsers validate a 0.5B model's output, raising exact-match accuracy from 13% (original design) to 52% with no crashes (Section 7.1).
- MedVoice-Eval, a new parallel test set of 25 medicine-reminder requests in Hindi, Tamil, Telugu and Bengali with gold labels, and a script that reproduces every number in this report (Sections 5.1, 10).
- An ablation and robustness study quantifying the contribution of translation, prompting, frequency rules and validation, the effect of the wrong source language and of speech-like noise (Section 7.3).
- An error analysis that identifies a reproducible clock-time bug in IndicTransToolkit preprocessing and drug-name transliteration as the dominant error sources (Section 7.4).

1.5 Organisation of the Report

Section 2 reviews recent work on voice and conversational agents for older adults, Indian-language speech and translation, and structured extraction with language models, and identifies

the gap. Section 3 sets measurable objectives. Section 4 describes the innovation, architecture and AI techniques, and Section 5 the data, models and implementation. Section 6 defines the evaluation protocol and Section 7 reports results, ablations and errors. Section 8 discusses responsible-AI issues, Section 9 concludes, and Sections 10–12 cover reproducibility, references and the declaration of generative-AI use.

2. Literature Review

2.1 Search Strategy

Sources: Google Scholar, Semantic Scholar, arXiv, ACL Anthology, ACM Digital Library and PubMed Central. Keyword groups: ("voice assistant" OR "conversational agent") AND ("older adults" OR elderly) AND (medication OR health); "medication adherence" AND (app OR mHealth) AND (review OR meta-analysis); (IndicTrans2 OR "Indian languages") AND (translation OR ASR); ("structured output" OR "guided generation" OR "information extraction") AND "large language model"; "small language model" AND on-device. Date range January 2023 – October 2026. Inclusion: peer-reviewed papers or clearly marked preprints with a DOI or arXiv ID, relevant to a system component or to the users. Exclusion: pre-2023 work, items without a verifiable identifier, blogs and product pages. About 45 results were screened by title and abstract and 14 papers were included; each was opened on its publisher, ACL Anthology, arXiv or PubMed Central page to confirm the bibliographic details and the numbers quoted.

2.2 Literature Summary Table

Table 2.1 summarises the 14 reviewed papers; Section 2.3 compares them.

Table 2.1: Summary of reviewed literature (14 papers, 2023–2025)

#	Authors, Year [Ref.]	Title and Venue (journal / conference / preprint)	Problem, Concept or Framework	Methodology / Implementation	Dataset and Evaluation Metrics	Key Findings (with numbers)	Limitations / Gaps Identified	Relevance to Our Project
1	Gala et al., 2023 [1]	IndicTrans2 — TMLR 2023	MT for all 22 scheduled Indian languages	Transformer MT trained on new BPCC corpus; distilled 200M variants	BPCC: 230M bitext pairs (644K human-translated); new n-way benchmark; chrF++	Surpasses prior open and commercial systems on existing and new benchmarks	Evaluated on general text, not short spoken commands; drug brand names not studied	Used as our Indic→English translator (indic-en-dist-200M)
2	Qwen Team (Yang et al.), 2024 [2]	Qwen2.5 Technical Report — arXiv preprint	Open LLM family from 0.5B to 72B	Pre-training on 18T tokens; SFT on 1M+ samples; multistage RL	Standard LLM benchmarks (MMLU, coding, maths, instruction following)	72B-Instruct competitive with Llama-3-405B-Instruct (~5× larger)	Little evidence on small models in non-English, domain-specific extraction	Qwen2.5-0.5B-Instruct is our extractor; 1.5B is a baseline
3	Javed et al., 2024 [3]	IndicVoices — Findings of ACL 2024	Inclusive speech data for Indian languages	Large-scale crowd collection across districts; IndicASR model	7,348 h from 16,237 speakers, 145 districts, 22 languages; 1,639 h transcribed	Built IndicASR, first ASR covering all 22 languages	ASR only; no downstream task such as command understanding	Motivates replacing browser ASR with Indic ASR (future work)
4	Bhogale et al., 2023 [4]	Vistaar — INTERSPEECH 2023	Benchmarking ASR for Indian languages	59 benchmarks; IndicWhisper fine-tuned on 10.7K h, 12 languages	WER on 59 language–domain benchmarks vs 3 public and 2 commercial systems	Lowest WER on 39/59 benchmarks; mean WER reduction 4.1	Older or accented elderly speech not separately analysed	Shows ASR error is unavoidable → we test robustness to noisy input
5	Etxaniz et al., 2024 [5]	Do Multilingual LMs Think Better in English? — NAACL 2024 (short)	Translate-test vs direct inference	"Self-translate": model translates input to English, then solves task	5 multilingual tasks; accuracy	Translating to English consistently beats direct non-English inference	Uses the model's own translation; does not test dedicated MT or Indic scripts	Justifies our English pivot with a dedicated MT model; we ablate it
6	Tam et al., 2024 [6]	Let Me Speak Freely? — EMNLP 2024 Industry Track	Effect of JSON/format restrictions on LLMs	Compare free-form vs JSON/XML/YAML-constrained outputs	Reasoning and classification benchmarks	Strict format constraints degrade reasoning; stricter = larger drop	Studies large models; little on small models	Supports loose JSON instructions + post-hoc validation instead of strict constrained decoding
7	Willard & Louf, 2023 [7]	Efficient Guided Generation for LLMs — arXiv preprint	Constrained decoding by regex/grammar	Generation as FSM transitions over a vocabulary index (Outlines)	Overhead of guided generation	Guarantees format with little overhead	Guarantees syntax, not correct values (e.g. a valid but wrong time)	Alternative to our validation layer; considered as future work
8	Xu et al., 2024 [8]	LLMs for Generative Information Extraction: A Survey — Frontiers of Computer Science	LLM-based IE taxonomy	Survey of IE subtasks and techniques (prompting, fine-tuning, ICL)	Survey of reported results across IE benchmarks	In-context examples and schema descriptions are key drivers of LLM IE quality	Mostly English and large models	Our task is slot-filling IE; we adopt schema-in-prompt + few-shot
9	Liu et al., 2024 [9]	MobileLLM — ICML 2024	Sub-billion LLMs for on-device use	Deep-and-thin architecture, weight sharing	Zero-shot reasoning, chat and API-calling benchmarks	125M/350M models +2.7%/+4.3% over prior SOTA; near LLaMA-2-7B on API calling	English-centric; device is a phone, not an end-to-end app	Supports choosing a sub-1B model for local, private inference

#	Authors, Year [Ref.]	Title and Venue (journal / conference / preprint)	Problem, Concept or Framework	Methodology / Implementation	Dataset and Evaluation Metrics	Key Findings (with numbers)	Limitations / Gaps Identified	Relevance to Our Project
10	Yang et al., 2024 [10]	Talk2Care — Proc. ACM IMWUT 8(2)	LLM voice assistant between older adults and providers	Interviews (10 older adults, 9 providers); LLM VA + provider dashboard	Two user studies; qualitative and time-saving measures	Enriched health information collected and saved provider time	English only; no medication scheduling; cloud LLM	Closest work on LLM voice assistants for older adults' health
11	Mahmood et al., 2025 [11]	Voice Assistants for Health Self-Management — CHI 2025	Co-designing an LLM VA with older adults	Interviews (N=17), co-design (N=10), in-home usability (N=5)	Qualitative; usability study	Older adults want personalised, routine-aware medication reminders and control	Small samples; English; no accuracy evaluation of extraction	Defines user needs our system targets; we add quantitative extraction evaluation
12	Mahmood et al., 2024 [12]	Situated Understanding of Errors in Older Adults' Interactions with VAs — arXiv preprint	Error patterns in real use	Month-long in-home study, 15 homes, ChatGPT-powered VA probe	Audio logs; error analysis	Errors are frequent; LLM context can help prevent and repair them	Preprint; English; small sample	Motivates our error-safe pipeline and clear error messages
13	An et al., 2025 [13]	Conversational Agents for Older Adults' Health: A Systematic Review — arXiv preprint	Acceptance of health CAs	Systematic literature review	72 papers	Low acceptance: unstable effects, independence and privacy concerns	Preprint; few studies in low-resource languages	Motivates local processing, user control and robustness
14	Lanke et al., 2025 [14]	Mobile Apps on Medication Adherence for Chronic Conditions — J. Med. Internet Res. 27	Do adherence apps work?	Systematic review and random-effects meta-analysis of RCTs	14 RCTs (2014–2022, n = 57–412); MMAS-8, MMAS-4, % adherence	MMAS-8 +0.57 (0.33–0.80); % adherence +18.85 (2.17–35.53); MMAS-4 n.s.	Apps heterogeneous; not language- or age-specific	Evidence that reminder apps are worth building

2.3 Synthesis and Research Gap

Older adults and health voice agents. Recent HCI work agrees that voice suits older adults and that medication reminders are a core need [10], [11], but also that real-world use breaks down through recognition and dialogue errors [12] and that acceptance is low when behaviour is unstable or privacy is unclear [13]. These studies are qualitative, English-only and built on cloud LLMs; none measures how accurately a request is turned into a schedule.

Indian-language speech and translation. Indian-language MT and ASR now cover all 22 scheduled languages [1], [3], [4], and translating into English before reasoning improves multilingual LLM performance [5]. These resources are evaluated on general text or speech, not on short, information-dense commands containing drug brand names and times, where a single mistranslated word changes the schedule.

Structured extraction with (small) language models. LLMs perform information extraction well when given a schema and in-context examples [8], and sub-billion models can run on device [2], [9]. Strict format constraints, however, can hurt model quality [6], and grammar-constrained decoding [7] guarantees syntax but not that the time or frequency is correct or executable. Evidence on small models doing slot extraction in a safety-relevant setting is scarce.

Gap. No prior work evaluates, end to end and per language, a locally running pipeline that turns Indian-language medicine-reminder requests into executable schedules, or quantifies how a symbolic validation layer compensates for a small model's errors. **Gap statement:** there is no measured, low-cost, multilingual and error-safe method for converting older adults' spoken medicine-reminder requests in Indian languages into correct reminders, which this project builds and evaluates.

3. Objectives of the Project

MedVoice is a voice-first web app: an older adult picks their language on a slider, speaks a request, and the backend translates it, extracts a structured reminder with a small local language model, validates it with deterministic rules, stores it in MongoDB and schedules a notification. Its distinguishing features are (a) four Indian languages through an English pivot, (b) a neuro-symbolic design in which the scheduler's own parser is the final judge of the model's output, and (c) fully local inference on an 8 GB laptop with no API cost. The objectives and targets in Table 3.1 were fixed before the evaluation was run.

Table 3.1: Project objectives and how they are measured

#	Objective (specific and measurable)	Measured by (metric / evidence)	Target or success criterion	Reported in Section
O1	Turn Indian-language reminder requests into correct, schedulable reminders	Exact-match accuracy (all four fields correct) on the 100-item test set	≥ 70%, and significantly higher than both baselines (exact McNemar, Holm-corrected $p < 0.05$)	7.1, 7.2
O2	Serve all four languages equally	Per-language exact-match accuracy; χ^2 test across languages	No language below 60%; no significant difference ($p \geq 0.05$)	7.1, 8.3
O3	Run interactively on an 8 GB laptop	End-to-end extraction latency (translation + LLM), mean and p95	p95 < 5 s per request; ₹0 API cost	5.6, 7.1
O4	Never fail unsafely	Share of requests returning a valid reminder or a clear error (no server crash); dose-hallucination rate	100% handled without a crash; dose hallucination < 10%	7.1, 7.4
O5	Quantify what each component contributes	Change in exact-match accuracy when each component is removed	Every component's contribution reported with a significance test	7.3

4. Innovation Component and System Design

4.1 Innovation Component

The innovative part of MedVoice is a **neuro-symbolic extraction pipeline in which a very small language model proposes and deterministic code disposes**. Qwen2.5-0.5B reads a messy translated sentence well, but writes times and repetition rules the scheduler cannot use ("night", "every day for the next 3 months", "not specified"). The validation layer therefore maps time words to clock times, recomputes the frequency from the model's answer and the translated sentence with pattern rules, and uses the scheduler's own parsers as the acceptance test, so a reminder is saved only if it can be executed; anything else returns a clear message to the user.

This is non-trivial because errors compound across three noisy stages (speech recognition, translation, a 0.5B model); because a safety-relevant task must fit an 8 GB memory budget where a larger model is not an option; and because translation and drug-name transliteration differ by language, so the design must be evaluated per language.

4.2 Positioning Against the Closest Existing Work

Table 4.1: Comparison with the three closest existing works

Feature / capability	Talk2Care [10]	Mahmood et al. VA [11]	Adherence apps (meta-analysis) [14]	Our project
Voice input for older adults	Yes	Yes	Mostly no (touch UI)	Yes
Indian languages	No (English)	No (English)	Not reported	Yes: Hindi, Tamil, Telugu, Bengali
Free-form request → structured schedule	No (collects health info)	Yes (reminders from visit notes)	No (manual entry)	Yes
Model runs locally / no API cost	No (cloud LLM)	No (cloud LLM)	n/a	Yes (0.5B + 200M on 8 GB laptop)
Validation of LLM output before acting	Not described	Not described	n/a	Yes (rules + scheduler parsers)
Quantitative extraction evaluation	No	No	No (adherence outcomes)	Yes (100 items, baselines, ablations)
Evaluation with older-adult users	Yes	Yes (N=5 in-home)	Yes (14 RCTs)	No (future work)

4.3 System Architecture

Fig. 4.1 shows the end-to-end architecture. The React front end captures speech with the browser's Web Speech API in the language chosen on the slider and sends the text and language code to the FastAPI back end (POST /sendData). The pipeline translates the text with IndicTrans2, extracts a JSON reminder with Qwen2.5-0.5B-Instruct, validates and normalises it and saves it in MongoDB; the scheduler then reads the stored document and creates APScheduler jobs. MongoDB is the scheduler's only source of reminders: on start-up and every 60 s the jobs are rebuilt to match the active reminders in the database. At the due time a job shows a native macOS notification. The front end lists reminders (GET /reminders) and can delete them (DELETE /reminders/{id}), which also cancels their jobs.

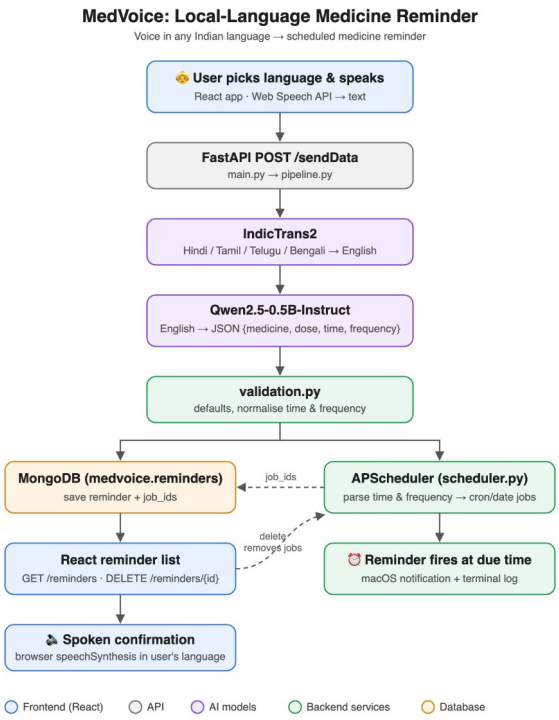


Fig. 4.1: MedVoice architecture — speech → IndicTrans2 → Qwen2.5-0.5B → validation → MongoDB + APScheduler → notification

4.4 Module Description

Front end (frontend/src/App.jsx; React 19 + Vite). Voice-first UI. Input: speech and the language chosen on the slider. Output: transcript sent to /sendData, reminder cards, spoken confirmation. Uses the browser Web Speech API for recognition and synthesis; retries loading reminders until the back end is ready and shows back-end errors.

API and pipeline (backend/main.py, services/pipeline.py; FastAPI). Runs translate → extract → validate → save → schedule on {message, language} and always returns {success, message}; any exception becomes a readable error, and a reminder that fails to schedule after saving is rolled back.

Translation (models/indicTrans2.py). Maps the app's language code to IndicTrans2's (hi-IN→hin_Deva, ta-IN→tam_Taml, te-IN→tel_Telu, bn-IN→ben_Beng) and translates with indictrans2-indic-en-dist-200M (IndicTransToolkit pre/post-processing, 5 beams).

Extraction (models/Qwen.py). Qwen2.5-0.5B-Instruct, float16, chat template with a format-defining system prompt and three examples, greedy decoding, ≤ 80 new tokens; the first JSON object in the reply is parsed. Prints an estimated and live time-to-finish.

Validation (services/validation.py). Coerces values to text, fills defaults ("1 tablet", "9:00 PM", "daily"), maps time words, recomputes frequency with rules and accepts the reminder only if the scheduler's parsers succeed.

Scheduler, notifications and storage (services/scheduler.py, connectToMongoDB.py). MongoDB stores each reminder and is the scheduler's single source of truth: on start-up and every 60 s, jobs are created, rescheduled or removed to match the active reminders, including edits made directly in the database. APScheduler cron jobs cover daily, weekday and bounded "next N days/months" reminders (counted from created_at); a macOS notification is shown via osascript at the due time.

All modules were built and evaluated by the sole team member, Laksh Ahuja, with the AI assistance declared in Section 12.

4.5 AI Techniques Used and Design Rationale

Task formulation

Three AI tasks are chained: speech recognition (browser, off the shelf) turns audio into text in the chosen language; machine translation maps it to English; and slot extraction maps the English sentence to medicine and dose (free text), time (clock time) and frequency (daily, every <weekday>, next N days, next N months). Validation then decides deterministically whether the reminder can be scheduled.

Level of autonomy: acts after human approval — the user sees the transcript and presses "Confirm reminder", and every saved reminder is visible and deletable; the model's output is used only if the symbolic checks accept it. **Where error enters:** misheard drug names or numbers, transliterated brand names, wrong or invented slot values, and defaults (an omitted dose becomes "1 tablet").

Techniques used

Table 4.2: AI techniques used in the project

AI technique	Used? (Y/N)	Level*	Where in our system, and why chosen over the main alternative
Large language models and the transformer architecture (attention, tokenisation, decoding and temperature)	Y	(b)	Qwen2.5-0.5B-Instruct (decoder-only transformer) extracts slots; IndicTrans2 (encoder–decoder transformer) translates. Greedy decoding chosen over sampling for repeatable outputs (Section 7.1).
Prompt engineering and in-context learning (zero-/few-shot, chain-of-thought, structured outputs)	Y	(b)	Chat-template system prompt defining each slot's format + 3 few-shot examples + JSON-only instruction; chosen over fine-tuning because no labelled training data exists and it costs nothing to change.

AI technique	Used? (Y/N)	Level*	Where in our system, and why chosen over the main alternative
Fine-tuning and parameter-efficient adaptation (instruction tuning, LoRA / QLoRA)	N		Not used: no training data; prompting + validation reached the target without it.
Embeddings and dense retrieval (vector search, similarity measures, hybrid search, re-ranking)	N		Not used: no document collection to search.
Retrieval-augmented generation (chunking, grounding, citation of sources)	N		Not used (future work: retrieve from a drug-name list to correct transliterations).
Knowledge graphs and GraphRAG (entity and relation extraction, graph traversal, KG-based validation)	N		Not used.
LLM agents: tool use, planning and memory (ReAct, function calling, MCP)	N		Not used: a fixed pipeline is more predictable than tool-calling agents for a safety-relevant task.
Multi-agent coordination (role specialisation, debate, voting, deliberation)	N		Not used.
Hallucination detection and fact verification (claim decomposition, self-consistency, evidence checking)	N		Dose hallucination is measured (Section 7) but not automatically detected; noted as future work.
Uncertainty quantification and calibration (confidence estimation, conformal prediction, calibration error)	N		Not used.
Fuzzy logic and neutrosophic reasoning (membership functions, rule inference, T/I/F components, ANFIS)	N		Not used.
Neuro-symbolic integration (rules or logic combined with neural models, symbolic justification)	Y	(c)	validation.py: rules map time words, recompute frequency from model output + source sentence, and use the scheduler's parsers as the acceptance test for the LLM's output. Implemented by us.
Bias and fairness measurement (counterfactual / paired prompting, group fairness metrics)	Y	(b)	Per-language accuracy on a parallel test set (same 25 requests in 4 languages) with a χ^2 test.
Bias mitigation and guardrails (debiasing, input/output filtering, constrained generation)	Y	(c)	Output guardrails: type coercion, defaults, schedulability check, rollback and clear error messages instead of acting on bad output.
LLM evaluation methodology (benchmarks, LLM-as-a-judge, human evaluation)	Y	(c)	Hand-labelled gold set, two baselines, ablations, bootstrap CIs, exact McNemar tests with Holm correction.
Adversarial attacks and defences on LLMs (jailbreaks, prompt injection, red-teaming)	N		Not studied (prompt-injection exposure discussed in Section 8.4).
Multimodal models (vision-language models, speech, multimodal fusion)	Y	(a)	Speech: browser Web Speech API for recognition and speechSynthesis for spoken confirmation, chosen because it is free and needs no model download; IndicASR [3] is the main alternative.
Generative image models (diffusion, text-to-image generation)	N		Not used.

AI technique	Used? (Y/N)	Level*	Where in our system, and why chosen over the main alternative
Synthetic media detection and provenance (deepfake detection, content credentials)	N		Not used.
Long-context efficiency (context compression, summarisation, token budgeting)	N		Not needed: inputs are one sentence.
Agent-based modelling and simulation (information diffusion on social networks, epidemic-style models)	N		Not used.
Sequential decision-making under uncertainty (reinforcement learning, MDPs, game-theoretic strategy)	N		Not used.
Few-shot learning and edge deployment (meta-learning, quantisation, on-device inference)	Y	(b)	Sub-1B model in float16 (~1 GB) running locally on an 8 GB laptop (Apple M2, MPS); chosen over 1.5B, which did not fit in memory without disk offload.
Classical machine learning and computer vision (CNNs, gradient boosting, feature engineering)	N		Regular-expression rule baseline only (Section 6.3).
Other (specify)	Y	(a)	Neural machine translation (IndicTrans2) as an English pivot, chosen over prompting Qwen in Indian languages directly (ablated in Section 7.3).

Level: **(a) used off-the-shelf (e.g., an API call or library default); **(b)** configured or tuned by us; **(c)** modified or implemented by us.*

Technique explanations

Transformer models and decoding

IndicTrans2 is an encoder–decoder transformer: the encoder reads the Indic sentence with self-attention and the decoder generates English while attending to it, keeping the 5 best partial translations (beam search). Qwen2.5-0.5B is a decoder-only transformer that predicts each token from the previous ones, $p(x_t \mid x_{<t}) = \text{softmax}(W \cdot h_t / T)$. We use greedy decoding (the most probable token, the limit $T \to 0$), so the same request always gives the same reminder; higher temperatures flatten the distribution and make outputs vary (measured in Section 7.1). Limitation: greedy output can still be confidently wrong, and a 0.5B model knows little about Indian drug brands.

Prompt engineering and in-context learning

The chat template carries a system message that defines every field's format, three worked examples and the user's sentence (Appendix A). We chose prompting over fine-tuning because no labelled training data exists and prompts can be changed instantly. Limitation: examples and prior knowledge can leak into answers, which we measure as dose hallucination.

Neuro-symbolic validation

The model proposes slot values and rules decide what is executed: time words map to fixed clock times, frequency is recomputed by regular expressions over the model's answer and the translated sentence, and a reminder is accepted only if the scheduler's parse_time and parse_frequency succeed. We chose this over grammar-constrained decoding [7] because a grammar guarantees syntax, not an executable value, while our checks can also use the source sentence. Limitation: the rules cover only the patterns we support and default unknown frequencies to daily.

Fairness measurement, guardrails, speech and edge deployment

The test set contains the same 25 requests in four languages, so per-language differences reflect language rather than content; languages are compared with a χ^2 test. Guardrails coerce any model output (non-JSON, numbers, lists) and either build a valid reminder or return a clear message, never an HTTP 500. Speech uses the browser Web Speech API with the selected locale (hi-IN, ta-IN, te-IN, bn-IN), which is free but, in Chrome, processed online; Section 7 approximates its errors with noisy input. Qwen2.5-0.5B in float16 needs about 1 GB and runs on the laptop GPU through PyTorch MPS; the 1.5B model (about 3 GB) did not fit beside the translator on 8 GB and swapped to disk.

5. Methodology and Implementation

5.1 Data

Table 5.1: Dataset description

Attribute	Details
Name and source (URL / citation)	MedVoice reminder-extraction test set (this work): evaluation/testset.json in the project repository
Licence and terms of use	Released with the project repository; contains no third-party text
Size (samples, classes, tokens, images, documents)	25 reminder requests × 4 languages = 100 items; 4 slots per item (400 slot labels); 5 frequency types (daily, weekday, next N days, next N months)
Collection method (public benchmark / scraped / synthetic / LLM-generated / human-collected)	Synthetic, written for this project: 25 English requests covering common Indian medicines, doses, clock and word times and all frequency types, each translated into Hindi, Tamil, Telugu and Bengali. Drafted with an AI assistant (Section 12); not produced or filtered by the models under test
Train / validation / test split and how it was made	Test only: the system is not trained. The test set and its gold labels were fixed before any configuration was run on it; the prompt's three few-shot examples are different sentences

Attribute	Details
Class balance or demographic composition	Frequency: 12 daily, 5 weekday, 6 next-N-days, 2 next-N-months per language; 18 of 25 requests state a dose; 4 languages × 25 (balanced)
Personal or sensitive data present? How handled?	None: no real patients or users; medicine names are common public brands

5.2 Preprocessing

No training data is used, so there is no cleaning, augmentation or de-duplication step. At inference time: (1) the IndicTransToolkit IndicProcessor normalises the Indic text (Unicode normalisation, script handling, protecting numbers and Latin tokens such as "mg" and "B12") before SentencePiece tokenisation and translation, and post-processes the English output; (2) the English sentence is wrapped in the chat template and tokenised with Qwen's tokenizer; (3) the first JSON object in the model's reply is extracted with a tolerant JSON decoder; (4) validation coerces types and normalises time and frequency (Section 4.4).

For scoring only, predicted and gold values are normalised: medicine names are lower-cased and stripped of generic words (tablet, capsule, syrup, …) and plural endings; doses have number words converted to digits and units unified ("two spoons" = "2 teaspoons" = "2 spoon"); times and frequencies are compared after parsing with the scheduler's parsers. Leakage check: none of the 25 test requests appears in the prompt; one test drug (paracetamol) also appears in a few-shot example, with a different dose, time and wording.

5.3 Models Used

Table 5.2: Model inventory

Model	Exact version / snapshot	Open / closed weights	Size (params)	Role in system	Access method and date
IndicTrans2 (Indic→En, distilled)	ai4bharat/indictrans2-indic-en-dist-200M, snapshot d66a212	Open (MIT)	~200M	Translator (all systems)	Local, Hugging Face Hub download, Oct 2026
Qwen2.5-0.5B-Instruct	Qwen/Qwen2.5-0.5B-Instruct, snapshot 7ae5576	Open (Apache-2.0)	0.49B	Extractor (ours)	Local, Hugging Face Hub, Oct 2026
Qwen2.5-1.5B-Instruct	Qwen/Qwen2.5-1.5B-Instruct, snapshot 989aa79	Open (Apache-2.0)	1.54B	Original pipeline baseline; scale comparison	Local, Hugging Face Hub, Oct 2026
Web Speech API (SpeechRecognition, speechSynthesis)	Browser built-in (Chrome)	Closed	—	Speech recognition and spoken confirmation in the app (not in the offline evaluation)	Browser API, 2026

5.4 Prompting, Fine-tuning and Retrieval Configuration

Prompting: zero-shot was the original design; the final system uses a chat-format prompt (tokenizer.apply_chat_template) with a system prompt that defines each field and its allowed formats, three worked examples given as previous user/assistant turns, and the user's translated sentence. The output is requested as a JSON object only; no chain-of-thought is used, because reasoning text would have to be stripped and, for a 0.5B model, adds latency without clear benefit. Structured output is enforced by validation rather than constrained decoding (Section 4.5). Full prompts are in Appendix A.

Fine-tuning: none. **Retrieval:** none. **Orchestration:** a fixed sequential pipeline (no agents).

5.5 Experimental Setup and Hyperparameters

Table 5.3: Hyperparameters and settings

Parameter	Value(s) tried	Final value	How chosen (grid / random / Bayesian / default / literature)
Decoding (Qwen)	Greedy; sampling T = 0.7 (top-p 1.0, top-k off)	Greedy	Greedy gives repeatable output; sampling measured for variance only
max_new_tokens (Qwen)	150 (original), 80	80	Answers average ~35 tokens; 80 leaves headroom and caps worst-case latency
Few-shot examples	0, 3	3	Ablation (Section 7.3)
Model size (Qwen)	0.5B, 1.5B	0.5B	Memory limit of 8 GB; comparison in Section 7.2
Precision	float32 (original), float16	float16	Halves memory; float32 1.5B did not fit
Beam size (IndicTrans2)	5	5	IndicTrans2 recommended default
Noise rate (robustness test)	5% of characters deleted	5%	Chosen to mimic dropped sounds in ASR output; fixed seed per item
Random seeds and number of runs	Greedy: 3 runs; sampling: seeds 0, 1, 2	—	Bootstrap CIs with 10,000 resamples (seed 0)

5.6 Compute and Cost

Item	Details
Hardware (CPU / GPU / edge device, RAM)	Apple MacBook Air, Apple M2 (8-core CPU, integrated GPU via PyTorch MPS), 8 GB unified memory, macOS 26.6; no discrete GPU

Item	Details
Training / indexing time	No training. Model download once (IndicTrans2 0.9 GB, Qwen 0.5B 1.0 GB, Qwen 1.5B 3.1 GB). Full evaluation (300 translations, ~1,300 language-model calls) ≈ 1.5 h
Inference latency (mean and p95)	Translation 0.61 s (p95 0.81 s); Qwen-0.5B 1.11 s (p95 1.26 s); extraction end-to-end 1.73 s (p95 1.97 s). Qwen-1.5B 10.06 s per request on the same machine (memory-bound)
API tokens consumed and approximate cost (₹ / $)	No API tokens: all models run locally. Cost ₹0 (electricity only)

5.7 Tools and Frameworks

Table 5.4: Tools used and justification

Tool / library	Version	Purpose in project	Why chosen over alternatives
PyTorch	2.14.1	Model inference on Apple GPU (MPS)	Standard back end for Hugging Face models
Hugging Face Transformers	4.51.3	Load and run Qwen and IndicTrans2	Supports both models and chat templates out of the box
IndicTransToolkit	1.1.1	Indic pre/post-processing for IndicTrans2	Official toolkit; required for correct tokenisation
FastAPI + Uvicorn	0.141.1 / 0.52.4	REST API	Lightweight, typed, async; simpler than Django for a JSON API
APScheduler	3.11.3	Wall-clock reminder jobs	Runs inside the Python process; no separate broker, unlike Celery
MongoDB + PyMongo	8.2.11 / 4.18.0	Store reminders and job IDs	Schema-free documents match the JSON reminders
React + Vite	19.2 / 8.2	Front end	Fast development; component state for voice UI
Web Speech API	Browser built-in	Speech recognition and synthesis	Free, no download; Indian-language locales
SciPy	1.17.1	Statistical tests	Exact binomial test (McNemar) and χ^2

5.8 Implementation, Screenshots and Demo

GitHub repository (mandatory): https://github.com/laksh-ahuja06/local-language-ai-chatbot Demo video (mandatory): [TO FILL] Live deployment (optional): https://huggingface.co/spaces/Lkkash/Local_language_elderly_reminders

The system is implemented as a React single-page app and a Python FastAPI back end in one repository. Fig. 5.1 shows the main screen: the language slider in the header, the microphone button, the live transcript with a Confirm button, and the processing and error messages. Fig. 5.2 shows the "Your schedule" panel listing reminders loaded from MongoDB, each with its time and frequency and a Delete button that also cancels the scheduled jobs. The evaluation in

Sections 6–7 is reproduced by evaluation/run_eval.py (README: "Running the project"; evaluation stages listed in the script header).

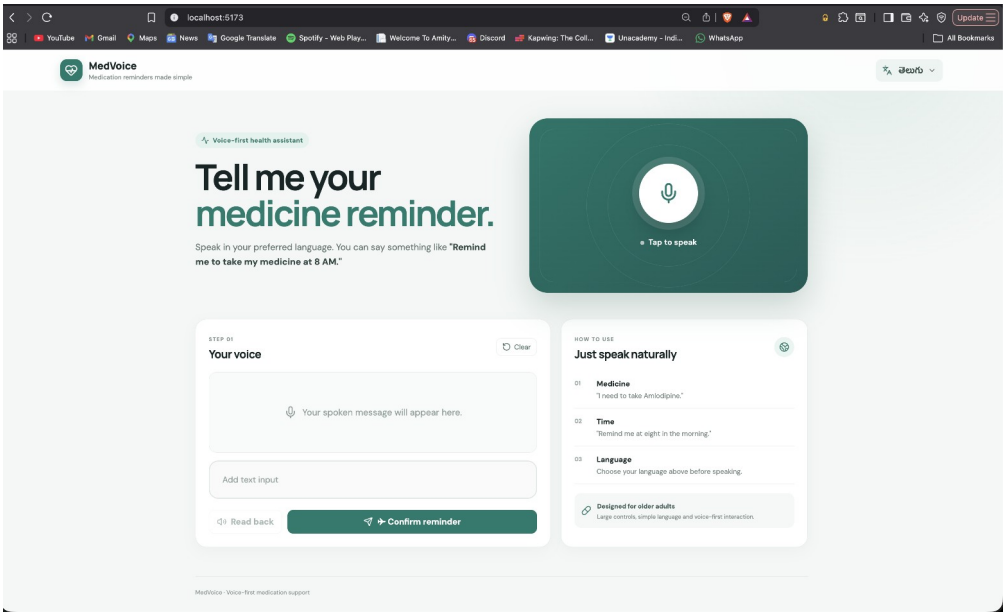


Fig. 5.1: Main screen — language slider, microphone and transcript confirmation

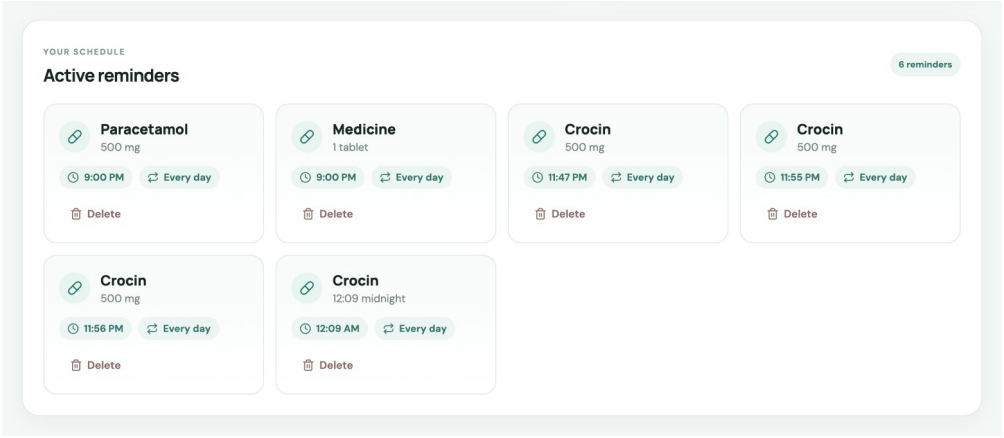


Fig. 5.2: "Your schedule" panel listing reminders stored in MongoDB

6. Evaluation Protocol

6.1 Evaluation Data and Split Hygiene

Evaluation uses the 100-item test set of Section 5.1 (25 requests × 4 languages). The items and gold labels were written and frozen before any configuration was run, and they were not used to design the prompt, whose examples are different sentences. Gold labelling rules (Appendix D): the medicine is the drug or product name (brand or generic, without words like "tablet"); the dose is what the user said, or empty if no dose was said; times are 24-hour clock times, with "at night" mapped to 21:00 as in the system's documented convention; frequency is daily unless a

weekday or a "next N days/months" duration is stated. Contamination: the requests are new, so they cannot be in the models' training data, although the drug names are public.

The evaluation starts from text, not audio: it measures translation + extraction + validation. Speech recognition is approximated by a noisy-input condition in which 5% of characters are deleted (fixed seed per item).

6.2 Metrics

Exact-match accuracy (primary): Acc = (1/N) Σ 1[medicine ∧ dose ∧ time ∧ frequency correct] after validation — a reminder is only useful if every field is right.

Per-field accuracy: medicine (exact after normalisation, plus a lenient version accepting string-similarity ≥ 0.8 to separate misspellings from wrong drugs), dose, time (parsed (hour, minute) equals gold) and frequency (parsed rule equals gold).

Schedulable rate: share of items whose time and frequency the scheduler can parse (task completion). **Dose-hallucination rate:** on the 28 items (7 requests × 4 languages) without a stated dose, the share where a dose other than empty or the default "1 tablet" is reported — the main safety risk. **JSON-valid rate:** share of replies containing a parseable JSON object.

Latency: seconds per item for translation and the language model (mean and 95th percentile). **Per-language accuracy** with a χ^2 test (fairness). Track F's SUS score and user study were not carried out (Section 7.6).

6.3 Baselines

B1 — Rule-based (simple baseline). IndicTrans2 translation followed by regular expressions and keyword rules, with no language model: a clock-time regex (e.g. "9:30 p.m."), time words, a dose regex (number + unit), the word after "take/of/drink" as the medicine, and the same frequency rules and validation as our system. This represents the "current practice" of a hand-written parser and tests whether an LLM is needed at all.

B2 — Original pipeline (strong baseline, established model). The project's first version: Qwen2.5-1.5B-Instruct (a 3× larger model) with the original zero-shot plain-text prompt, max 150 new tokens, and the original validation that only fills defaults ("night", "everyday") without normalising or checking. Crashes on non-JSON output are counted as failures. This is the same task without our key components.

We also report **Ours with Qwen-1.5B** (same prompt and validation, larger model) to separate the effect of model size from the effect of the prompt and validation.

6.4 Human Evaluation and LLM-as-a-Judge (if used)

No human evaluation of outputs and no LLM-as-a-judge were used: all metrics are computed automatically against the gold labels with deterministic normalisation rules (Appendix D), so no judge validation is required. Gold labels were written by the author, with AI assistance (Section 12), from the English source request; the Tamil, Telugu and Bengali translations have not been checked by native speakers, which is a limitation (Section 7.6).

6.5 Statistical Analysis

Greedy decoding is deterministic, so the deployed configuration was run three times to confirm identical outputs; to measure sensitivity to decoding randomness it was additionally run three times with sampling ($T = 0.7$, seeds 0–2), and the mean ± standard deviation across these runs is reported. 95% confidence intervals for accuracies are percentile bootstrap intervals over items (10,000 resamples). Paired comparisons between our system and each baseline or ablation on the same items use the exact McNemar test (binomial test on discordant pairs); because many comparisons are made, p-values are corrected with the Holm–Bonferroni method. Differences between languages are tested with a χ^2 test of independence.

7. Results and Discussion

7.1 Main Results

Table 7.1: Main results on the 100-item test set (accuracy %, 95% bootstrap CI for exact match; best in bold). Greedy decoding was identical in 3 runs (SD 0); sampling at T = 0.7 is shown separately

Method	Exact match (all 4 fields)	Per-field accuracy: medicine / dose / time / frequency	Schedulable / dose halluc.	LLM latency (mean)
Baseline 1 — rule-based (no LLM)	42% [32, 52]	52% / 80% / 83% / 100%	88% / 0%	—
Baseline 2 — original pipeline (Qwen-1.5B, zero-shot)	13% [7, 20]	74% / 46% / 24% / 68%	34% / 86%	16.25 s
Ours — Qwen-0.5B + chat prompt + validation	52% [42, 62]	74% / 76% / 83% / 100%	100% / 39%	1.11 s
Ours with Qwen-1.5B	61% [51, 71]	74% / 88% / 83% / 100%	96% / 0%	10.06 s

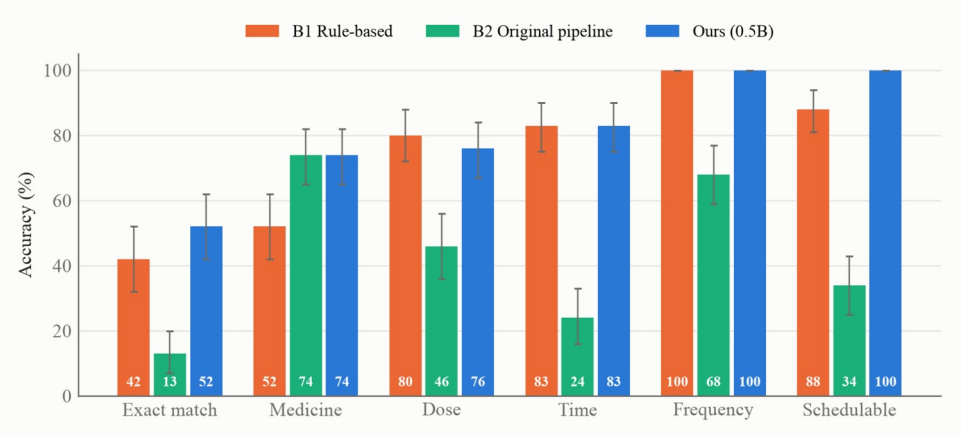


Fig. 7.1: Per-field accuracy of the two baselines and our system (error bars: 95% bootstrap CI)

Table 7.1 and Fig. 7.1 show that our system produced a fully correct reminder for 52% of the 100 requests (95% CI [42, 62]), against 42% for the rule-based parser and 13% for the original 1.5B pipeline. Frequency was always correct and every request was schedulable, because both are enforced by the validation layer; the remaining errors are in the medicine name (74% correct, 92% if near-miss spellings are accepted), the dose (76%) and the time (83%). The dose-hallucination rate was 39% on the 28 items without a stated dose. Greedy decoding gave identical outputs in three runs (identical); with sampling at T = 0.7 the exact-match accuracy was 52.0 ± 2.6% (mean ± SD over seeds 0–2: 49%, 53%, 54%), confirming that greedy decoding is the right choice for a reminder system.

Extraction took 1.73 s on average (p95 1.97 s; translation 0.61 s + model 1.11 s), well within the 5 s target; the 1.5B model needed 10.1 s because it does not fit in 8 GB beside the translator.

7.2 Comparative Analysis

Against the rule-based baseline our system was right on 21 items where the rules failed and wrong on 11 where they succeeded (exact McNemar p = 0.110, Holm-adjusted p = 0.220): the rules match regular time and dose patterns well once translated, but often pick the wrong words as the medicine (52% vs 74%).

Against the original pipeline (B2) it won on 39 items and lost on 0 (Holm-adjusted $p < 0.001$) with a model three times smaller; B2's answers were often unexecutable (34% schedulable) because its prompt defined no formats and its validation kept values such as "night". The same prompt and validation with Qwen-1.5B reached 61% [51, 71] (Holm-adjusted p = 0.035 vs 0.5B) at 9× the latency on this hardware: the prompt and validation, not model size, drive the improvement over the original design.

No published system reports extraction accuracy for Indian-language medicine reminders, so a numeric comparison with the literature is not possible. Qualitatively, the dominance of translation (Section 7.3) agrees with the advantage of translating into English reported in [5], and the value of defining the output schema in the prompt agrees with [8].

7.3 Ablation Study

Table 7.2: Ablation — effect of removing each component

Configuration	Exact match (95% CI)	Change vs full system (McNemar, Holm-adjusted p)
Full system (Qwen-0.5B, chat + 3 examples, v2 validation)	52% [42, 62]	—
– without few-shot examples (system prompt only)	53% [43, 63]	+1 pts (p = 1.000)
– without system prompt and examples (original plain prompt)	13% [7, 20]	-39 pts (p < 0.001)
– without rule-based frequency (trust Qwen's frequency)	40% [31, 50]	-12 pts (p = 0.002)
– without validation layer (original defaults-only validation)	40% [31, 50]	-12 pts (p = 0.002)
– without IndicTrans2 (Indic text straight to Qwen)	0% [0, 0]	-52 pts (p < 0.001)
Wrong source language (all inputs translated as Tamil)	17% [10, 25]	-35 pts (p < 0.001)
Noisy input (5% characters deleted)	35% [26, 44]	-17 pts (p < 0.001, unadjusted)

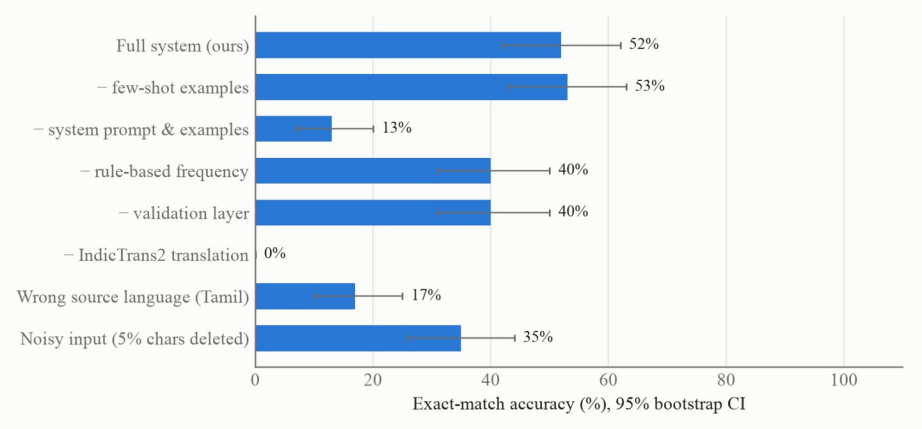


Fig. 7.2: Ablation and robustness — exact-match accuracy when each component is removed or the input is degraded

Translation is essential: without it the model produced no correct reminder (0%), and translating from the wrong language — the original hard-coded Tamil setting — gave 17%, which justifies the language slider. **The format-defining system prompt** comes next: the original plain prompt gave 13%, with 71% valid JSON and 23% correct times. **The rule-based frequency step** added 12 points (p = 0.002); replacing the whole validation layer with the original one gave the same 40%, so the frequency rules carry the accuracy effect and the rest of

validation carries the safety effect. Contrary to H2, **the few-shot examples did not help** (53% without them, $p = 1.000$).

Robustness: deleting 5% of input characters reduced accuracy to 35% ($p < 0.001$) and the rule-based baseline to 31%, mainly through damaged times and medicine names.

7.4 Error Analysis

Our system failed on 48 of 100 items; 31 of these failures involve a single field. Table 7.3 categorises them and the cases below show representative failures with their diagnosis.

Table 7.3: Categorised errors of our system (Qwen-0.5B)

Error type (our system, 48 failed items; an item can have several)	Items	Example
Medicine name misspelt by transliteration (near miss, similarity ≥ 0.8)	18	Glycomet → "glycomate"; cetirizine → "saturizine"
Medicine: wrong span or wrong word	8	"ORS packet"; Dolo → "TOLO 650"; Montair → "Monterey pill"
Dose invented when none was said	11	Crocin → "500 mg"; Dolo 650 → "10 units"
Dose wrong (number in brand name, wrong unit, time in dose field)	13	Shelcal 500 → "500 tablets"; ORS → "1 tablet"
Time corrupted by translation-toolkit bug (H:MM → "1:MM")	12	7:30 PM → "1:30 PM"; 9:45 PM → "1:45 PM"
Time word mapped differently ("night")	4	"at night" → 11:00 PM instead of 9:00 PM
Other time error	1	"at 4 p.m." → 12:00 PM (hi #22)

Case 1 — time corrupted in translation (12 items, all languages). Hindi #3 "शाम 7:30 बजे मेटफॉर्मिन की दो गोलियां…" → "…metformin at 1: 30 p.m." → 1:30 PM (expected 7:30 PM). IndicTransToolkit 1.1.1 rewrites "7:30" as the literal "\1◌:30" (an unfilled regex back-reference; colon turned into a visarga), which the translator renders as "1". Two-digit hours (11:15) are protected and survive. A library bug, fixed after the evaluation by padding the hour (Section 7.7).

Case 2 — drug name transliterated (18 near misses). Bengali #25 "…সেটিরিজিন খাওয়ার…" → "…eat satirizine…" → medicine "saturizine" (expected cetirizine); time and frequency correct. Phonetic spellings of brands are re-spelt by the translator and the model has no drug list to correct them.

Case 3 — invented dose (11 items). Hindi #1 "मुझे हर रोज़ रात 9 बजे क्रोसिन लेने की याद दिलाना" (no dose) → dose "500 mg". The 0.5B model fills the slot from prior associations despite the instruction; the 1.5B model never did. The most safety-relevant failure.

Case 4 — strength in a brand name read as a count. Tamil #23 "…ஒரு ஷெல்கால் 500 மாத்திரை…" → "A Shelcal 500 tablet…" → dose "500 tablets" (expected 1 tablet). Shows why the parsed reminder must be read back to the user.

Case 5 — time word ignored. Telugu #6 "రాత్రి ఒక ఆస్పిరిన్ మాత్ర…" → "…aspirin pill at night" → 11:00 PM (expected 9:00 PM by the documented convention). The model converted "night" itself; validation only maps the word when it is returned unchanged.

Safe rejection. With Qwen-1.5B, all four versions of request #22 gave the impossible time "16:00 PM"; validation rejected them with an error message instead of saving a wrong reminder.

7.5 Answers to the Research Questions

RQ1. 52% fully correct (95% CI [42, 62]): significantly better than the original pipeline (13%, Holm-adjusted $p < 0.001$) but not than the rule-based parser (42%, $p = 0.220$). H1 is partly supported; the 70% target was not met.

RQ2. Translation matters most (-52 points without it), then the format-defining prompt (-39) and the frequency rules (-12); the few-shot examples made no difference (+1). H2 holds for translation, not for the examples.

RQ3. Accuracy was equal across languages (χ^2 $p = 1.00$), supporting H3, but below the 60% target of O2; noise cost 17 points and the wrong source language 35.

RQ4. 0.5B was 9 points below 1.5B (52% vs 61%, $p = 0.035$) but 9× faster, supporting H4; 1.5B also invented no doses (0% vs 39%), so it is the safer choice where memory allows.

7.6 Threats to Validity and Limitations

Internal validity. The test sentences, their translations and gold labels were drafted with an AI assistant (Section 12) and checked only by the author, not by native speakers of every language. Scoring choices (strict medicine match, "night" = 21:00) affect the numbers; a lenient medicine metric shows their effect. The same 25 requests appear in four languages, so items are not independent and the bootstrap intervals are probably too narrow. The 1.5B latency was measured under memory pressure.

External validity. Evaluation starts from text, not older adults' speech; recognition errors, accents and code-mixing are only approximated by character deletion. Four languages, 25 request templates, common brands and one laptop were tested; complex requests (several medicines or times) were not.

Construct validity. Exact match treats a misspelt drug like a wrong drug and ignores error severity. No user study, task-completion test or System Usability Scale was run, so usability and effect on adherence are not shown — the main gap against Track F's requirements.

7.7 Post-Evaluation Fix (reported separately)

After the evaluation, the largest error source from Section 7.4 was addressed: (1) indicTrans2.py pads single-digit hours before translation ("8:03" → "08:03", also for Indian-script digits), which IndicTransToolkit then protects correctly; (2) validation.py reads an explicit clock time from the translated sentence instead of trusting the model. Translation and Qwen-0.5B were re-run on the same 100 items, all else unchanged (evaluation/run_postfix.py).

Table 7.4: Post-evaluation fix on the same test set (not a held-out result)

Configuration (Qwen-0.5B unless stated)	Exact match (95% CI)	Medicine / dose / time / frequency	Schedulable
Original system (Section 7.1, before the fix)	52% [42, 62]	74% / 76% / 83% / 100%	100%
+ time-safe translation	57% [47, 67]	74% / 76% / 93% / 100%	98%
+ time-safe translation + sentence time rule (current system)	57% [47, 67]	74% / 76% / 96% / 100%	100%
B1 rule-based with time-safe translation	44% [34, 54]	52% / 80% / 99% / 100%	100%

Time accuracy rose from 83% to 96% and exact match from 52% to 57% (5 items gained, 0 lost; Holm-adjusted $p = 0.141$) — not significant, because most of these requests also had a wrong medicine or dose. The time rule left exact match unchanged here but raised time accuracy from 93% and made every output schedulable; it also fixes time-only requests where the model dropped the time. Versus the rule-based baseline with the same fix (44%): 25 wins, 12 losses, Holm-adjusted $p = 0.141$. Languages stayed balanced (χ^2 $p = 0.989$).

Caveat: the fixes were designed after seeing errors on this test set, so Table 7.4 is optimistic; Sections 7.1–7.3 remain the reported evaluation and a new held-out set is needed. The remaining 43 failures are mostly drug names (medicine 74% correct) and doses (76%; dose hallucination 36%).

8. Responsible AI and Ethical Considerations

8.1 Risk Assessment

Table 8.1: Risks and mitigations

Risk (e.g., misuse, bias, privacy, over-reliance, hallucination)	Who could be harmed	Likelihood / severity	Mitigation implemented
Wrong reminder (wrong time, frequency or drug) from mis-recognition, mistranslation or model error	Older adult taking medicine	Medium / High	Validation rejects unexecutable output; transcript shown and confirmed before saving; reminder cards visible and deletable; per-field accuracy measured
Invented dose (model fills a dose the user never said)	Older adult	Low–medium / High	Dose hallucination measured; the notification says what the user said; planned: show the parsed reminder and ask for explicit confirmation of the dose
Over-reliance: missed reminder while the backend is not running	Older adult	Medium / High	Jobs are rebuilt from MongoDB on start-up and re-synced every 60 s, so restarts lose nothing; reminders due while the backend is stopped are still missed (documented)
Privacy: voice and medication data	Users	Medium / Medium	Translation and extraction run locally; data stays in a local MongoDB; browser speech recognition may use an online service (disclosed)
Unequal quality across languages	Speakers of lower-performing languages	Medium / Medium	Per-language evaluation reported (Section 8.3); language chosen explicitly on the slider rather than guessed
Prompt injection through the spoken text	System integrity	Low / Low	The model has no tools; output is limited to four validated text fields that are only used to create a reminder

8.2 Data Privacy and Consent

The system handles health-related personal data (medicines, doses, schedules and the spoken sentence). Translation, extraction and storage run on the user's own machine with no third-party AI API; the exception is the browser's Web Speech API, which may send audio to an online service (Google's, in Chrome) and must be disclosed or replaced by local Indic ASR before deployment. No real patient data was collected and no user study was run, so no consent was needed. Under the Digital Personal Data Protection Act, 2023 [18], a deployment would need notice and consent for processing health data, a way to view and erase it (the app already lists

and deletes reminders) and security safeguards (the local MongoDB currently has no authentication).

8.3 Fairness

Language is the group attribute most likely to affect this system, so the test set is parallel: the same 25 requests in each language. Table 8.2 reports exact-match accuracy per language.

Table 8.2: Exact-match accuracy by language (25 items each)

Language	Ours (95% CI)	B1 rule-based	B2 original
Hindi	52% [32, 72]	52%	12%
Tamil	52% [32, 72]	44%	16%
Telugu	52% [32, 72]	36%	16%
Bengali	52% [32, 72]	36%	8%

Our system scored 52% in every language (χ^2 $p = 1.00$), although different items failed in each; the rule-based baseline was less even (Hindi 52%, Tamil 44%, Telugu 36%, Bengali 36%). With 25 items per language the intervals are about ±20 points. Accents, dialects, code-mixed speech, speaker age and gender, and speech or hearing impairments were not tested and are left for a user study.

8.4 Safety, Security and Misuse

MedVoice is a research prototype that reminds people of medicines already prescribed to them. It gives no medical advice, does not check doses or interactions, and is not a substitute for a doctor or pharmacist; it does not handle emergencies, which a deployment should route to a human contact or helpline. Guardrails: validation, confirm-before-save, visible and deletable reminders and clear error messages. Prompt injection is low-risk because the model has no tools or data access and its output is limited to four validated fields.

8.5 Licences and Attribution

Reused components and their licences: IndicTrans2 models and IndicTransToolkit — MIT; Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct — Apache 2.0; PyTorch — BSD-3-Clause; Hugging Face Transformers — Apache 2.0; FastAPI — MIT; APScheduler — MIT; React and Vite — MIT; MongoDB Community Server — SSPL; PyMongo — Apache 2.0. The evaluation data was written for this project. The repository does not yet contain a LICENSE file; an MIT licence is recommended.

9. Conclusion and Future Work

MedVoice shows that a voice-first, four-language medicine-reminder assistant can run entirely on an 8 GB laptop. Against the objectives: **O1 not met** — the system produced fully correct reminders for 52% of requests, significantly better than the original design (13%) but short of the 70% target and not significantly better than a rule-based parser (42%); **O2 partly met** — accuracy was identical across Hindi, Tamil, Telugu and Bengali but below 60% in each; **O3 met** — extraction took 1.7 s on average (p95 2.0 s) at no API cost; **O4 partly met** — every request returned a valid reminder or a clear error with no crash, but the 0.5B model invented a dose in 39% of requests that had none; **O5 met** — every component's contribution was measured.

The main lesson is that, for a small model in a safety-relevant task, the surrounding engineering matters more than the model: translation and a format-defining prompt turned 0% and 13% into 52%, rules made every output executable, and the largest remaining errors come from the translation toolkit and drug-name spelling rather than from the language model's reasoning.

After the evaluation, the clock-time bug was fixed by padding single-digit hours before translation, which raised time accuracy to 96% and exact match to 57% on the same (already-seen) test set (Section 7.7). Remaining future work, in order of expected impact: (1) match medicine names against a list of Indian brand and generic names with fuzzy matching, which the lenient metric suggests could raise medicine accuracy from 74% towards 92%; (2) accept a dose only if a number or unit appears in the source sentence, and use the 1.5B model where memory allows, to remove invented doses; (3) read the parsed reminder back to the user and require explicit confirmation; (4) replace browser speech recognition with a local Indic ASR model [3]; and (5) evaluate with older adults on real speech, with a System Usability Scale and task-completion study, using a larger test set checked by native speakers. Each of these must be evaluated on new test data, since the current set has now been used for analysis.

10. Reproducibility Checklist

Item	Yes / No / NA	Location (file or section)
Repository shared with the faculty; README has setup and run steps	Yes (README); sharing [TO FILL]	README.md ("Running the project")
Dependencies pinned (requirements.txt / environment.yml / package.json)	Partly	backend/requirements.txt (only transformers pinned); exact versions in Table 5.4; frontend/package.json

Item	Yes / No / NA	Location (file or section)
Exact model identifiers and experiment dates recorded	Yes	Table 5.2 (snapshots); experiments run 15–16 Oct 2026
All prompts included (Appendix A and repository)	Yes	Appendix A; backend/models/Qwen.py; evaluation/run_eval.py
Random seeds fixed and reported	Yes	Section 5.5; evaluation/run_eval.py
Evaluation set or script to regenerate it provided	Yes	evaluation/testset.json
One script reproduces the main results table	Yes	evaluation/run_eval.py all → evaluation/results/tables.md
Data sources and licences listed	Yes	Sections 5.1 and 8.5
No API keys, passwords or personal data in the repository	Yes	backend/services/.env holds only a localhost MongoDB URI (should be replaced by .env.example)
Compute and API cost reported	Yes	Section 5.6
Demo video link opens without a permission request	[TO FILL]	Section 5.8

11. References

[1] J. Gala, P. A. Chitale, R. AK, V. Gumma, S. Doddapaneni et al., "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," Transactions on Machine Learning Research, 2023, arXiv:2305.16307.

[2] Qwen Team: A. Yang et al., "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[3] T. Javed, J. Nawale, E. George, S. Joshi et al., "IndicVoices: Towards building an inclusive multilingual speech dataset for Indian languages," in Findings of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, 2024, pp. 10740–10782, arXiv:2403.01926.

[4] K. S. Bhogale, S. Sundaresan, A. Raman, T. Javed, M. M. Khapra, and P. Kumar, "Vistaar: Diverse benchmarks and training sets for Indian language ASR," in Proc. INTERSPEECH 2023, arXiv:2305.15386.

[5] J. Etxaniz, G. Azkune, A. Soroa, O. Lopez de Lacalle, and M. Artetxe, "Do multilingual language models think better in English?" in Proc. 2024 Conf. North American Chapter of the ACL: Human Language Technologies (Volume 2: Short Papers), Mexico City, Mexico, 2024, pp. 550–564, doi: 10.18653/v1/2024.naacl-short.46.

[6] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen, "Let me speak freely? A study on the impact of format restrictions on large language model performance," in Proc. 2024 Conf. Empirical Methods in Natural Language Processing: Industry Track, Miami, FL, USA, 2024, arXiv:2408.02442.

[7] B. T. Willard and R. Louf, "Efficient guided generation for large language models," arXiv preprint arXiv:2307.09702, 2023.

[8] D. Xu, W. Chen, W. Peng, C. Zhang et al., "Large language models for generative information extraction: A survey," Frontiers of Computer Science, 2024, doi: 10.1007/s11704-024-40555-y.

[9] Z. Liu, C. Zhao, F. Iandola, C. Lai et al., "MobileLLM: Optimizing sub-billion parameter language models for on-device use cases," in Proc. 41st Int. Conf. Machine Learning (ICML), 2024, arXiv:2402.14905.

[10] Z. Yang, X. Xu, B. Yao, S. Zhang, E. Rogers, S. Intille, N. Shara, G. G. Gao, and D. Wang, "Talk2Care: An LLM-based voice assistant for communication between healthcare providers and older adults," Proc. ACM Interact. Mob. Wearable Ubiquitous Technol., vol. 8, no. 2, 2024, doi: 10.1145/3659625.

[11] A. Mahmood, S. Cao, M. Stiber, V. N. Antony, and C.-M. Huang, "Voice assistants for health self-management: Designing for and with older adults," in Proc. CHI Conf. Human Factors in Computing Systems (CHI '25), 2025, doi: 10.1145/3706598.3713839.

[12] A. Mahmood, J. Wang, and C.-M. Huang, "Situated understanding of errors in older adults' interactions with voice assistants: A month-long, in-home study," arXiv preprint arXiv:2403.02421, 2024.

[13] J. An, S. Yi, Y. Lyu, H. Liu, and Y. Zhang, "Conversational agents for older adults' health: A systematic literature review," arXiv preprint arXiv:2503.23153, 2025.

[14] V. Lanke, K. Trimm, B. Habib, and R. Tamblyn, "Evaluating the effectiveness of mobile apps on medication adherence for chronic conditions: Systematic review and meta-analysis," J. Med. Internet Res., vol. 27, e60822, 2025, doi: 10.2196/60822.

[15] AI4Bharat, "indictrans2-indic-en-dist-200M" model card, Hugging Face. [Online]. Available: https://huggingface.co/ai4bharat/indictrans2-indic-en-dist-200M (accessed 16 Oct. 2026).

[16] Qwen, "Qwen2.5-0.5B-Instruct" model card, Hugging Face. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct (accessed 16 Oct. 2026).

[17] Mozilla, "Web Speech API," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API (accessed 16 Oct. 2026).

[18] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023.

12. Declaration of Generative AI Use

This section covers AI tools used to **produce** the project and report (coding assistants, writing tools, literature-search assistants, image generators). AI models that are **components of your system** are described in Section 5.3, not here.

Table 12.1: Generative AI tools used

Tool and version	Purpose	Files / sections affected	Extent (minor edits / substantial draft / generated code)	How the output was verified
ChatGPT (OpenAI; version [TO FILL])	[TO FILL] (e.g. initial code, ideas, debugging)	[TO FILL]	[TO FILL]	[TO FILL]
Claude Code (Anthropic, model Claude Opus 5.5), October 2026	Debugging; code for validation, error handling, Qwen prompt and ETA, language slider, reminder cards, notifications, MongoDB-driven scheduler and time fix; evaluation script, test set and translations; literature search; drafting this report	backend/ (main.py, models/, services/); frontend/src/; evaluation/; README.md; all report sections	Substantial (generated code and draft text)	Code run and tested (unit checks, browser checks, evaluation runs); references opened at source; numbers from evaluation scripts. [TO FILL]: author to review all text; native speakers to check test-set translations

I confirm that I have reviewed, tested and understood all AI-assisted content; that every reference was checked against its original source; and that I take full responsibility for the accuracy and integrity of this report and code.

Appendices

Appendix A — Full prompts

A.1 System prompt of the deployed system (backend/models/Qwen.py, SYSTEM_PROMPT), sent through the Qwen chat template:

```
You extract medicine reminders from a user's message and reply with ONLY a JSON
object, no other text.

Fields:
- "medicine": the medicine name only, e.g. "Crocin", "Dolo 650", "vitamin D".
- "dose": amount and unit, e.g. "500 mg", "2 tablets", "1 capsule", "10 units",
"half tablet". Use "" if not mentioned.
- "time": clock time in the format "H:MM AM" or "H:MM PM", e.g. "9:00 PM", "7:30
AM". Convert words: morning = "8:00 AM", afternoon = "2:00 PM", evening = "6:00
PM", night = "9:00 PM".
- "frequency": exactly one of:
    "daily"
    "every monday" ... "every sunday"
    "next N days"     (e.g. "next 5 days")
    "next N months"   (e.g. "next 3 months")
  If no frequency is mentioned, use "daily". If a duration is mentioned ("for the
next 3 months"), use the "next N ..." form.

Never invent information that is not in the message.
```

A.2 Few-shot examples (FEW_SHOT_EXAMPLES), sent as earlier user/assistant turns before the user's translated sentence:

```
User: Remind me to take 2 tablets of Paracetamol at 10 pm every day
Assistant: {"medicine": "Paracetamol", "dose": "2 tablets", "time": "10:00 PM",
"frequency": "daily"}
User: I take Azithromycin 250 mg every Friday morning
Assistant: {"medicine": "Azithromycin", "dose": "250 mg", "time": "8:00 AM",
"frequency": "every friday"}
User: Remind me about my cough syrup at 8:30 pm for the next 4 days
Assistant: {"medicine": "cough syrup", "dose": "", "time": "8:30 PM", "frequency":
"next 4 days"}
```

A.3 Original zero-shot plain prompt (baseline B2 and the "plain prompt" ablation), sent as raw text without the chat template:

```
Extract the medicine reminder information from the user message.

User message:
{translated sentence}

Return ONLY valid JSON.
Do not include any explanation or extra text.

Use exactly these fields:
{
    "medicine": "",
    "dose": "",
    "time": "",
    "frequency": ""
}
```

No LLM-as-a-judge prompt was used (Section 6.4).

Appendix B — Additional results

Table B.1 lists every evaluated configuration (100 items each). Per-item predictions for all configurations are in evaluation/results/per_item.csv and raw model outputs in evaluation/results/qwen_0.5B.json and qwen_1.5B.json.

Table B.1: All configurations — accuracy (%) on the 100-item test set

Configuration	Exact	Med.	Med. (lenient)	Dose	Time	Freq.	Sched.	Dose halluc.
B1 Rule-based (IndicTrans2 + regex, no LLM)	42%	52%	64%	80%	83%	100%	88%	0%
B2 Original pipeline (Qwen-1.5B, plain prompt, v1 validation)	13%	74%	92%	46%	24%	68%	34%	86%
Ours (Qwen-0.5B, chat + few-shot, v2 validation)	52%	74%	92%	76%	83%	100%	100%	39%
Ours with Qwen-1.5B	61%	74%	92%	88%	83%	100%	96%	0%
- few-shot examples (system prompt only)	53%	77%	97%	70%	86%	100%	100%	43%
- system prompt and examples (plain prompt)	13%	35%	43%	60%	23%	100%	77%	14%
- rule-based frequency	40%	74%	92%	76%	83%	74%	74%	39%
- validation layer (v1 defaults only)	40%	74%	92%	76%	83%	74%	74%	39%
- IndicTrans2 (Indic text straight to Qwen)	0%	14%	15%	42%	42%	47%	97%	68%
Wrong source language (all translated as Tamil)	17%	29%	38%	51%	51%	73%	91%	54%
Ours on noisy input (5% characters deleted)	35%	61%	79%	68%	64%	93%	96%	32%
B1 on noisy input	31%	43%	56%	75%	70%	93%	85%	0%
Ours run 2 (greedy)	52%	74%	92%	76%	83%	100%	100%	39%
Ours run 3 (greedy)	52%	74%	92%	76%	83%	100%	100%	39%
Ours sampled T=0.7 seed 0	49%	73%	90%	72%	78%	100%	96%	43%
Ours sampled T=0.7 seed 1	53%	73%	92%	81%	83%	96%	99%	36%
Ours sampled T=0.7 seed 2	54%	74%	90%	81%	79%	100%	100%	25%

Run-to-run variability: three greedy runs gave identical outputs (52%, 52%, 52% exact match); three sampled runs at T = 0.7 gave 49%, 53%, 54% (mean 52.0%, SD 2.6 points).

Appendix C — User-study instruments and consent form

Not applicable: no user study was conducted (Section 7.6).

Appendix D — Annotation guidelines and labelling examples

Gold labels (evaluation/testset.json) were written from the English source request before any system was run, using these rules:

- Medicine: the product or drug name as said (brand or generic), without form words such as tablet, capsule or syrup; alternatives listed when equally correct ("Dolo 650" or "Dolo"; "B12" or "vitamin B12").
- Dose: the amount the user said with its unit ("500 mg", "2 tablets", "half tablet", "10 units"); if no amount is said, the gold dose is empty and only an empty value or the system default "1 tablet" is correct.
- Time: 24-hour [hour, minute]; part-of-day words follow the system's documented convention (morning 08:00, afternoon 14:00, evening 18:00, night 21:00).
- Frequency: daily unless a weekday ("every Monday") or a duration ("for the next 5 days/3 months") is stated; "every morning/night/day" counts as daily.
- Each request is translated into Hindi, Tamil, Telugu and Bengali keeping the same information; numbers are written with digits (Bengali items use Bengali digits) and units such as "mg" are kept in Latin script, as speech-recognition output usually shows them.

Example: "I have to take 10 units of insulin every day at 8 pm for the next 3 months" → medicine: insulin; dose: 10 units; time: [20, 0]; frequency: next 3 months.